

Fixed Parameter Complexity

Advanced Algorithms



Fixed parameter complexity

- Analysis what happens to problem when some parameter is *small*
- Today:
 - Definitions
 - Fixed parameter tractability techniques
 - Branching
 - Kernelisation
 - Other techniques

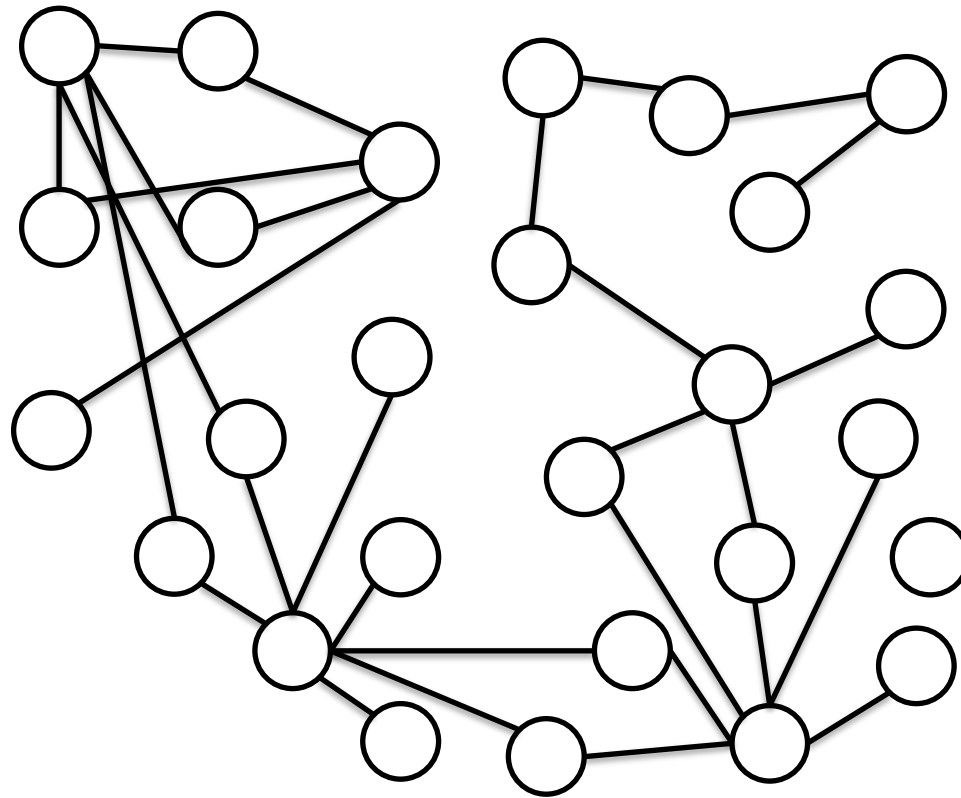


Motivation

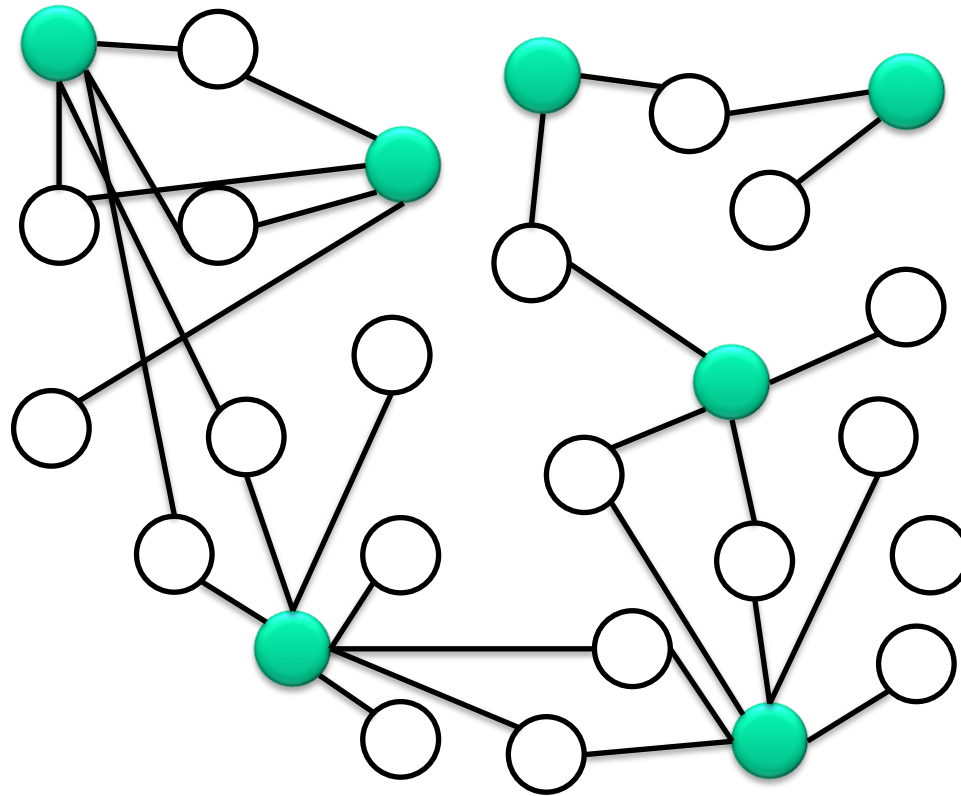
- Often, size of problem is gigantic
- In many applications, some aspect/number may be assumed to be *small*
 - We call this a *parameter*
- Time of algorithm can be exponential in this small number, but should be polynomial in *usual* size of problem



Example: Can you pick you pick
7 vertices that hit all edges?



Example: Can you pick you pick
7 vertices that hit all edges?



Parameterized graph problem

- **Given:** Graph G , integer k , ...
- **Parameter:** k
- **Question:** Does G have a ??? of size at least (at most) k ?
 - Examples: vertex cover, independent set, coloring, ...
- *Standard parameterization* of graph problem



Examples of parameterized problems (1)

Graph Coloring

Given: Graph G , integer k

Parameter: k

Question: Is there a vertex coloring of G with k colors? (I.e., $c: V \rightarrow \{1, 2, \dots, k\}$ with for all $\{v, w\} \in E: c(v) \neq c(w)$?)

- NP-complete, even when $k = 3$.

Examples of parameterized problems (2)

Clique

Given: Graph G , integer k

Parameter: k

Question: Is there a clique in G of size at least k ?

- Solvable in $O(n^{k+1})$ time with simple algorithm. Complicated algorithm gives $O(n^{2.23k/3})$. Seems to require $\Omega(n^{f(k)})$ time...



Examples of parameterized problems (3)

Vertex cover

Given: Graph G , integer k

Parameter: k

Question: Is there a vertex cover of G of size at most k ?

- Solvable in $O(2^k (n + m))$ time (and even slightly faster)



Fixed parameter complexity theory

- To distinguish between behavior:
 - $O(f(k) * n^c)$
 - $\Omega(n^{f(k)})$
- Proposed by Downey and Fellows
- Some call it multivariate complexity theory



Parameterized problems

- Instances of the form (x, k)
 - I.e., we have a *second parameter*
- Decision problem (subset of $\{0,1\}^* \times \mathbf{N}$)



Fixed parameter tractable problems

- **FPT** is the class of problems with an algorithm that solves instances of the form (x, k) in time $p(|x|) * f(k)$, for polynomial p and some function f
- FPT = fixed parameter tractable



Hard problems

- Complexity classes
 - $\mathbf{FPT} \subseteq \mathbf{W}[1] \subseteq \mathbf{W}[2] \subseteq \dots \mathbf{W}[i] \subseteq \dots \subseteq \mathbf{W}[P]$
 - FPT is ‘easy’, all others ‘hard’
 - Defined in terms of *Boolean circuits*
 - Problems **hard** for $\mathbf{W}[1]$ or larger classes are assumed not to be in FPT
 - Compare with P / NP



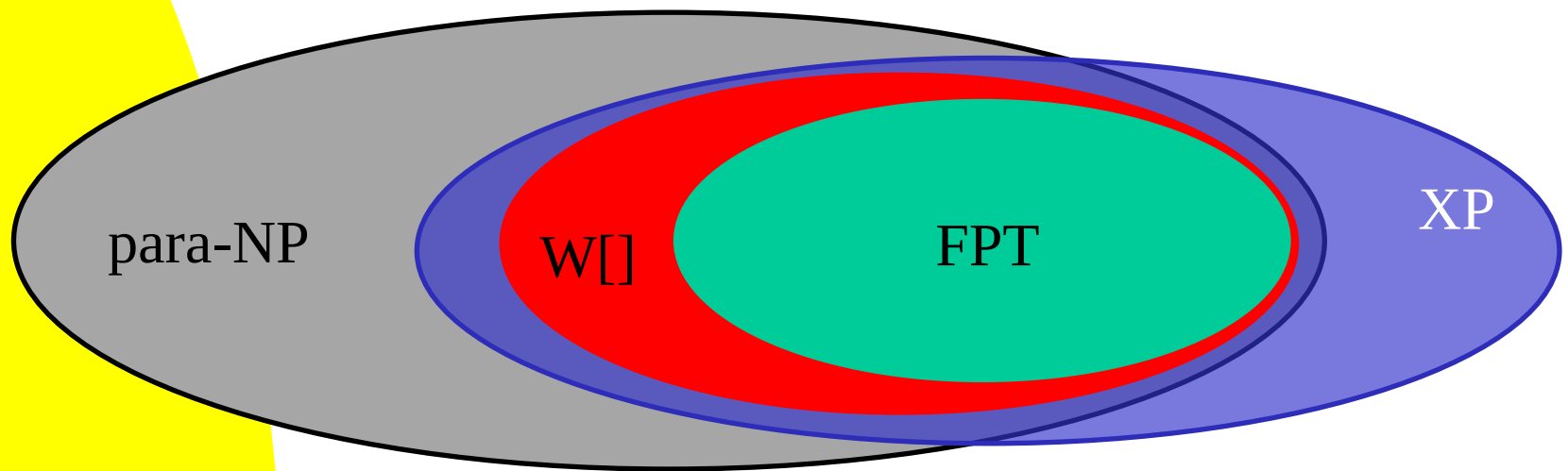
Examples of hard problems

- Clique and Independent Set are $W[1]$ -complete
- Dominating Set is $W[2]$ -complete
- This version of Satisfiability is $W[1]$ -complete
 - **Given**: set of clauses, k
 - **Parameter**: k
 - **Question**: can we set (at most) k variables to **true**, and all others to **false**, and make all clauses true?
- All have $n^{f(k)}$ -time algorithms
- Class of problems with $n^{f(k)}$ -time algorithms: XP



Even harder

- k -Coloring is NP-complete for $k=3$
- We call this *para-NP-complete*



Techniques for showing fixed parameter tractability

- Branching
- Kernelisation
- Other techniques (e.g., treewidth, Iterative compression)



Vertex Cover

BRANCHING ALGORITHM 1



A branching algorithm for vertex cover

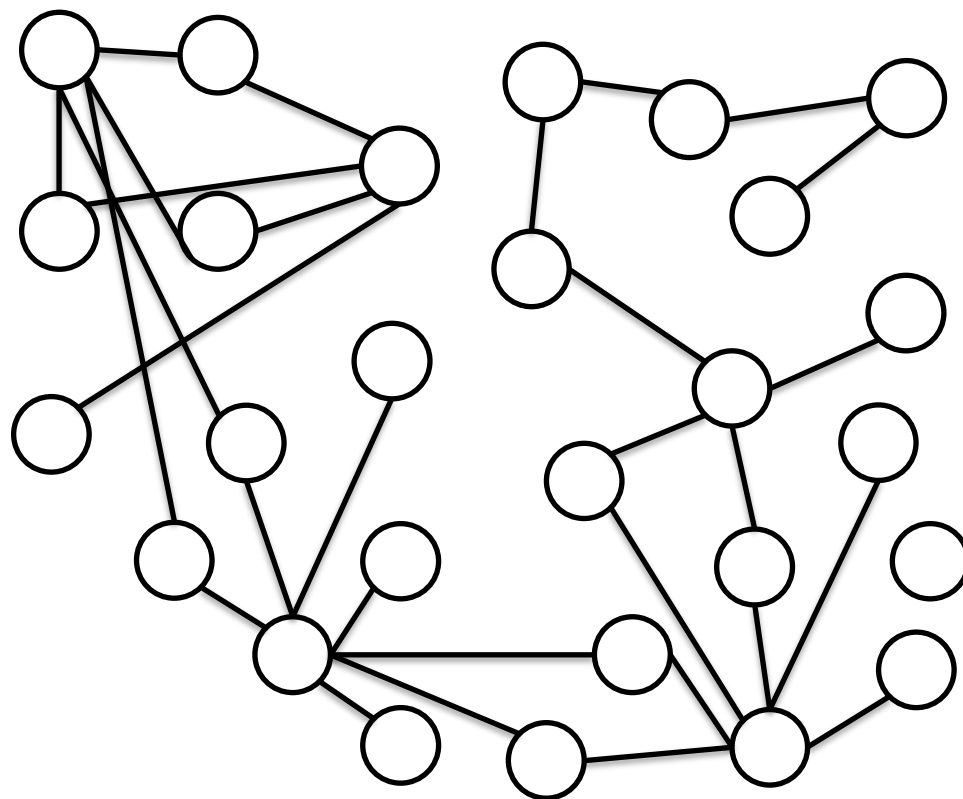
- Input: graph G and integer k
- Idea:
 - Simple base cases
 - Branch on an edge: at least one of the endpoints belongs to the vertex cover



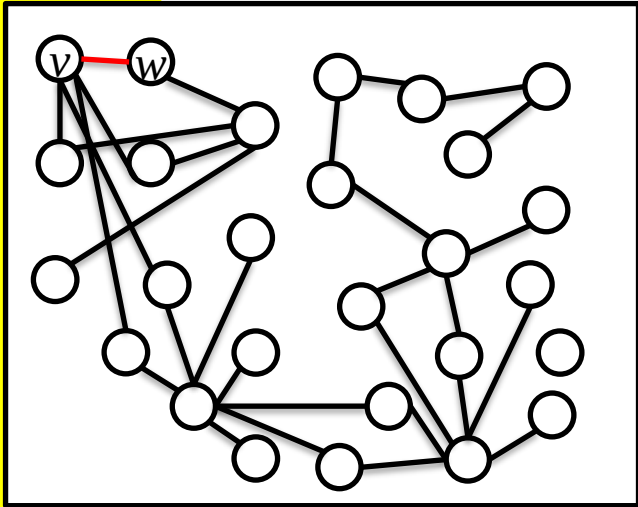
A branching algorithm for vertex cover

- Recursive procedure $\text{VC}(\text{Graph } G, \text{int } k)$
- $\text{VC}(G = (V, E), k)$
 - If G has no edges (and $k \geq 0$), return **true**
 - If $k == 0$, then return **false**
 - Select an edge $\{v, w\} \in E$
 - Compute $G' = G [V - v]$
 - Compute $G'' = G [V - w]$
 - Return $\text{VC}(G', k-1)$ or $\text{VC}(G'', k-1)$

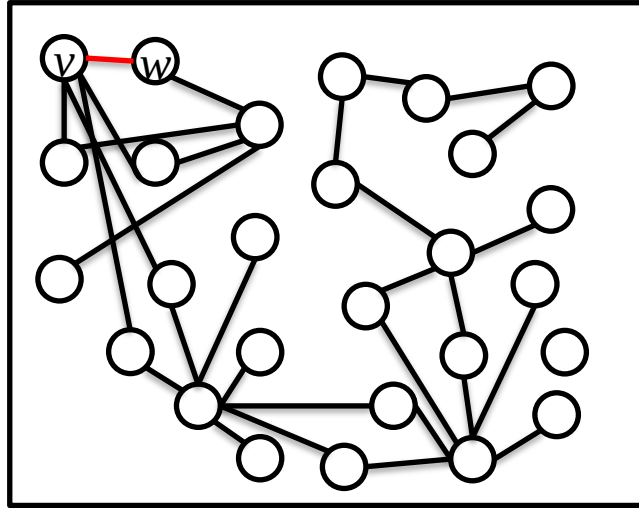




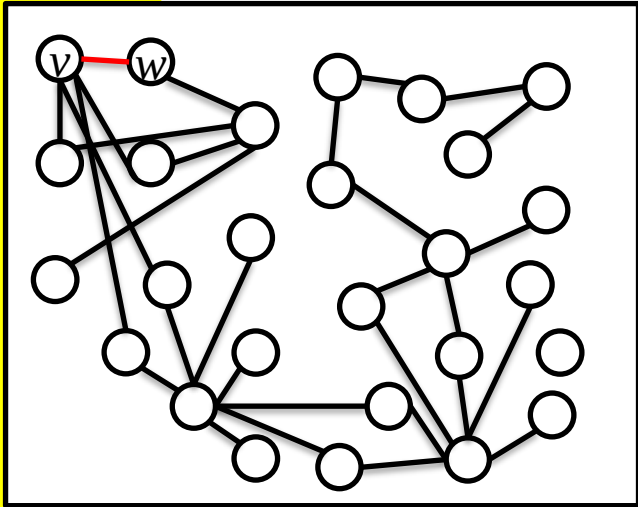
7



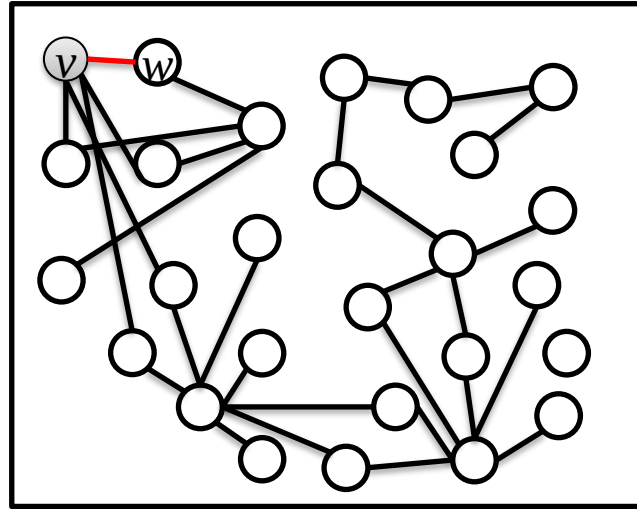
6



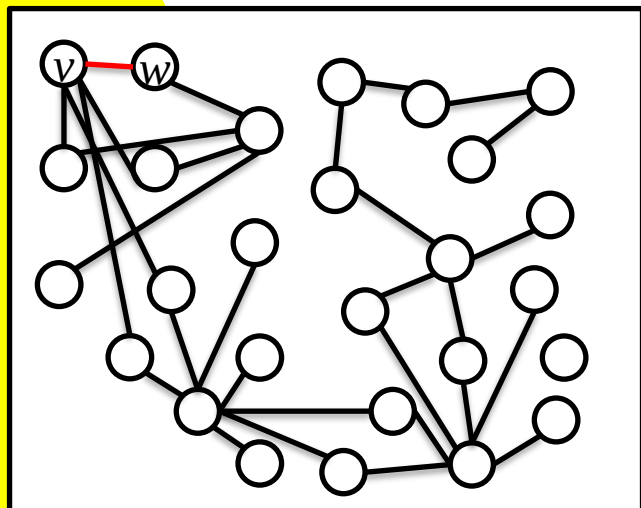
7



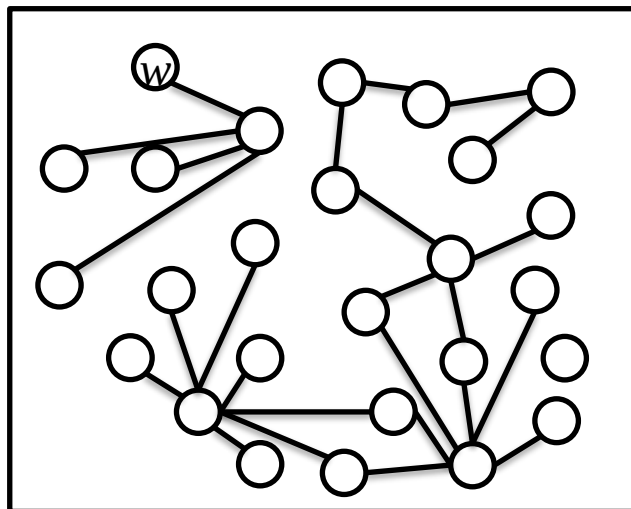
6



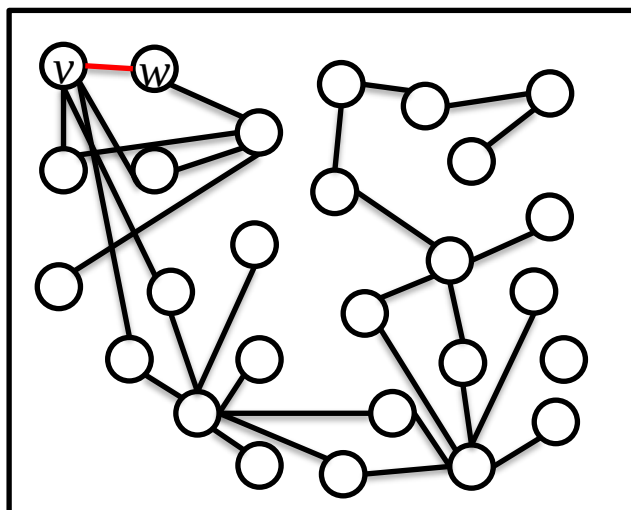
7



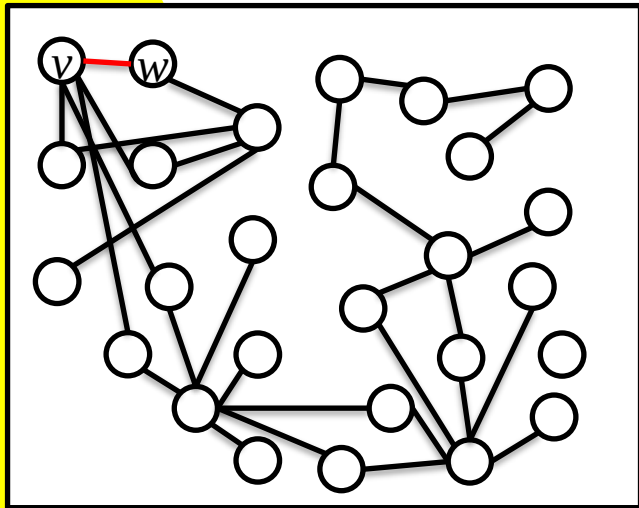
6



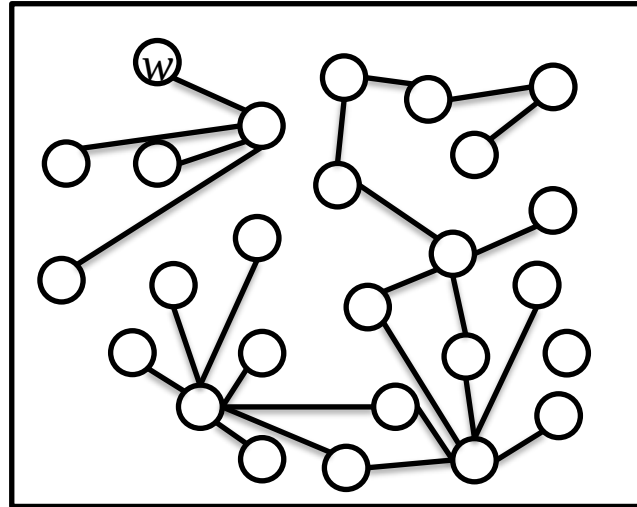
6



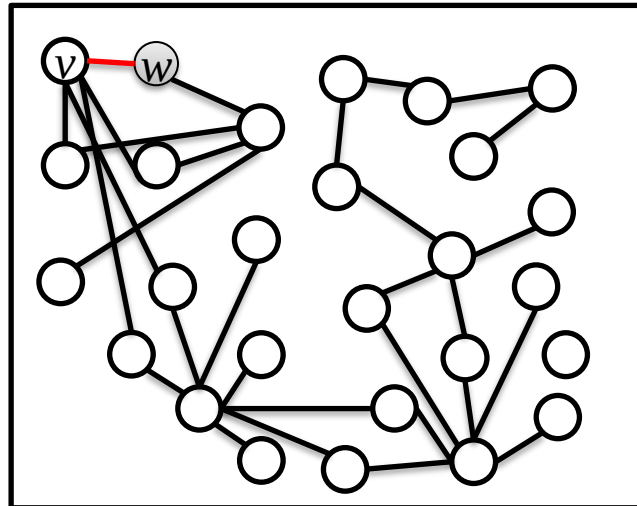
7



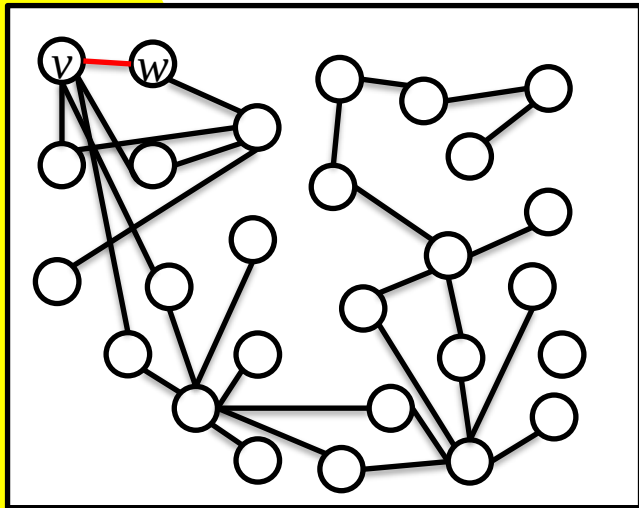
6



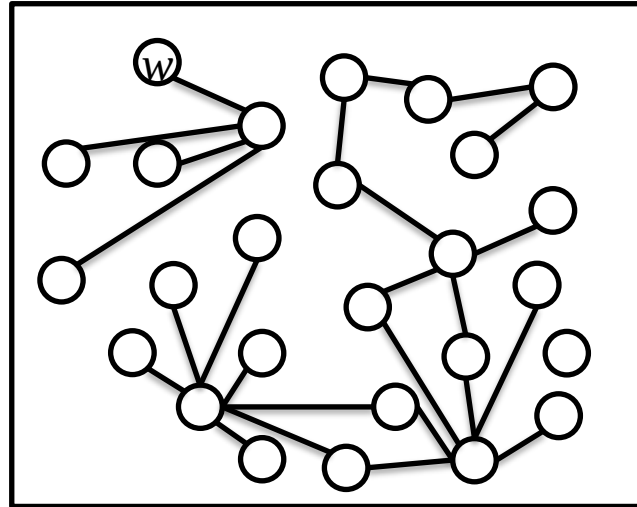
6



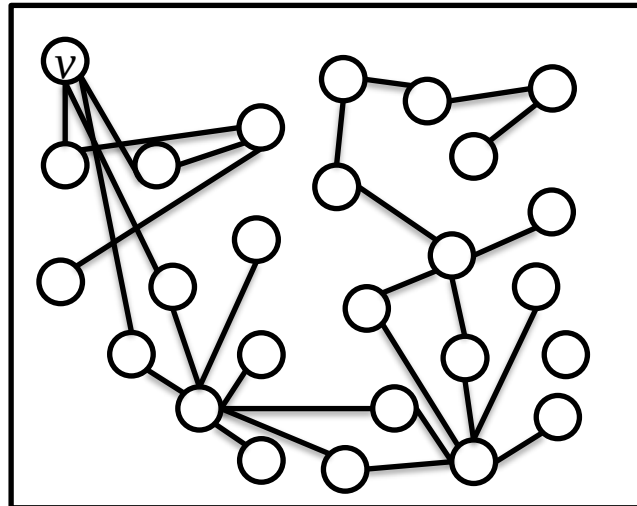
7



6



6



Analysis of algorithm

- Correctness
 - At least one of v , w belongs to an optimal VC
 - In a Yes-instance, at least one sequence of branches leads to the optimal solution
- Time analysis
 - Branching tree has 2^k leaves, so 2^k recursive calls
 - Each recursive call costs $O(n + m)$ time
 - $O(2^k k(n + m))$ (even $O(2^k (n + m))$) time: FPT



Unusual history of Vertex Cover

- $O(f(k) n^3)$ [Fellows Langston 1988]
 - f comes from Robertson & Seymour graph minors theorem (1983-2004)
- $O(kn + 2^k k^{2k+2})$ [Buss Goldsmith 1993]
- $O(3^k n)$ [Papadimitriou Yannakakis 1996]
- $O(2^k (n+m))$ [Monien *via* Mehlhorn 1984], [Downey Fellows 1992]



History of Vertex Cover

- $O(kn + 1.32472^k k^2)$ [Balasubramanian Fellows Raman 1998]
- $O(kn + 1.31951^k k^2)$ [Downey Fellows Stege 1998]
- $O(kn + 1.29175^k k^2)$ [Niedermeier Rossmanith 1999]
- $O(kn + 1.2852^k)$ [Chen Kanj Jia 2001]
- $O(kn + 1.2745^k k^4)$ [Chen Kanj Xia 2010]

<http://fpt.wikidot.com/fpt-races>

Cluster editing

BRANCHING ALGORITHM 2



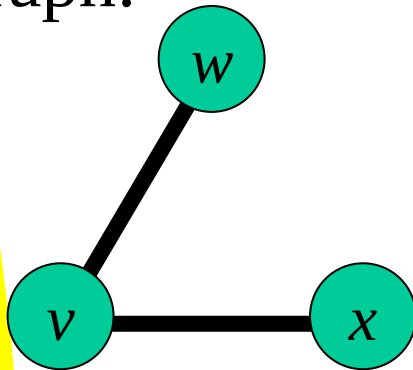
Cluster editing

- **Instance:** undirected graph $G = (V, E)$, integer k
- **Parameter:** k
- **Question:** can we make at most k modifications to G , such that each connected component is a clique, where each modification is an addition of an edge or the deletion of an edge?
- Models biological question: partition species in families, where available data contains mistakes
- NP-complete. Branching: $O(3^k p(n))$ algorithm



Lemma

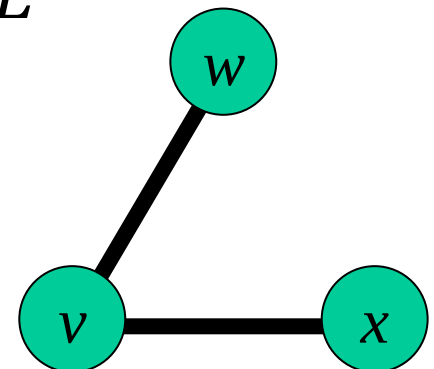
- If G has a connected component that is not a clique, then G contains the following subgraph:



- Proof: there are vertices w and x in the connected component that are not adjacent. Take such w and x of minimum distance, which must be 2.

Branching algorithm for Cluster Editing

- If each connected component is a clique:
 - Answer YES
- If $k=0$ and some connected components are not cliques:
 - Answer NO
- Otherwise, there must be vertices v, w, x with $\{v, w\} \in E$, $\{v, x\} \in E$, and $\{w, x\} \notin E$
 - Go three times in recursion:
 - Once with $\{v, w\}$ removed and $k = k - 1$
 - Once with $\{v, x\}$ removed and $k = k - 1$
 - Once with $\{w, x\}$ added and $k = k - 1$



Analysis branching algorithm

- Correctness by lemma
- Time analysis: branching tree has 3^k leaves



More on cluster editing

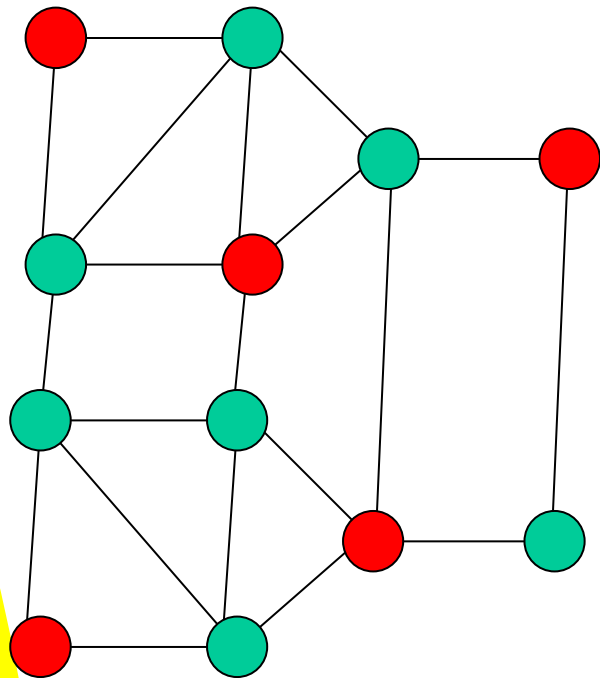
- Faster branching algorithms exist
- Important applications and practical experiments
- We'll see more when discussing kernelisation

Max SAT

- Variant of CNF-satisfiability, but now we ask: can we satisfy at least k clauses?
- NP-complete
- With k as parameter: FPT
- Branching:
 - Take a variable
 - If it only appears positively, or negatively, then ...
 - Otherwise: Branch! What happens with k ?

Independent Set on Planar Graphs

- **Given:** a **planar** graph $G = (V, E)$, integer k
- **Parameter:** k
- **Question:** Does G have an *independent set* of at least k vertices, i.e., a set W of size at least k , such that for all $v, w \in W$: $\{v, w\} \notin E$?



The red vertices
form an independent set

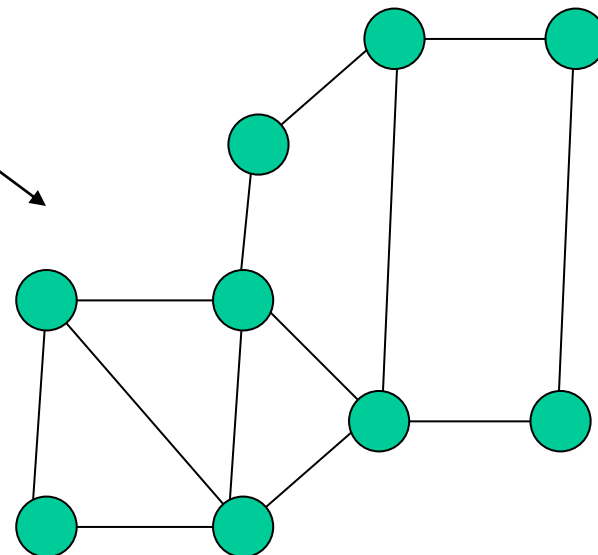
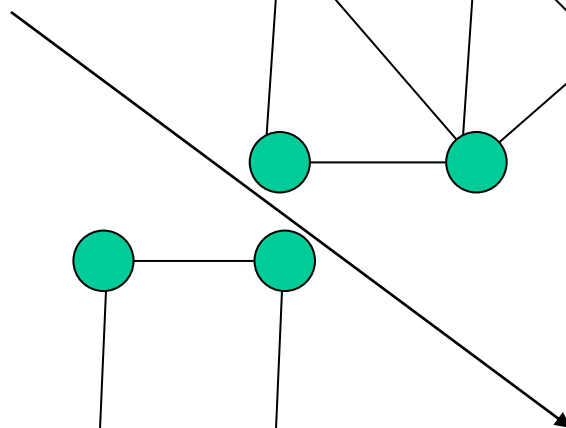
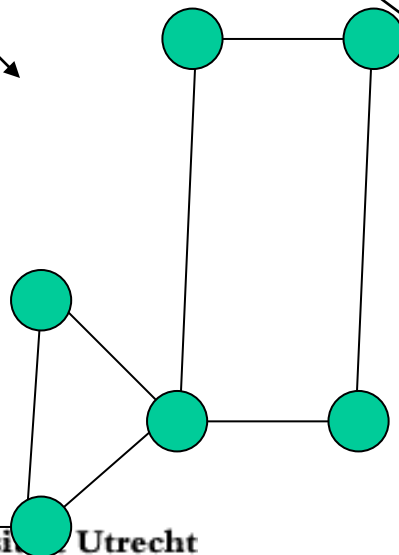
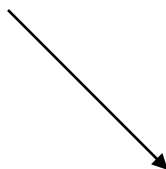
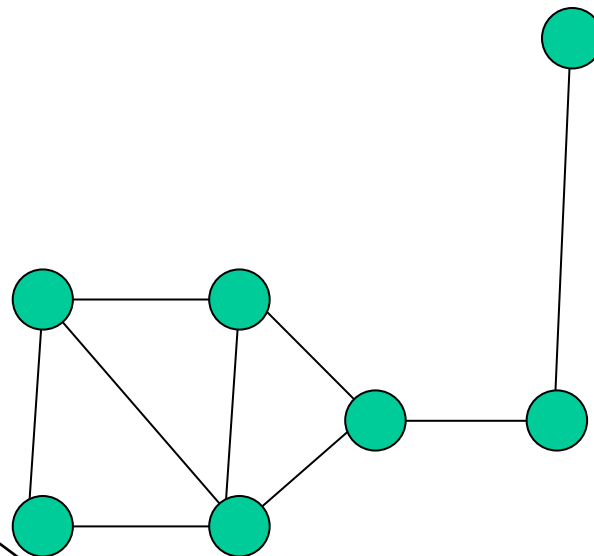
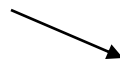
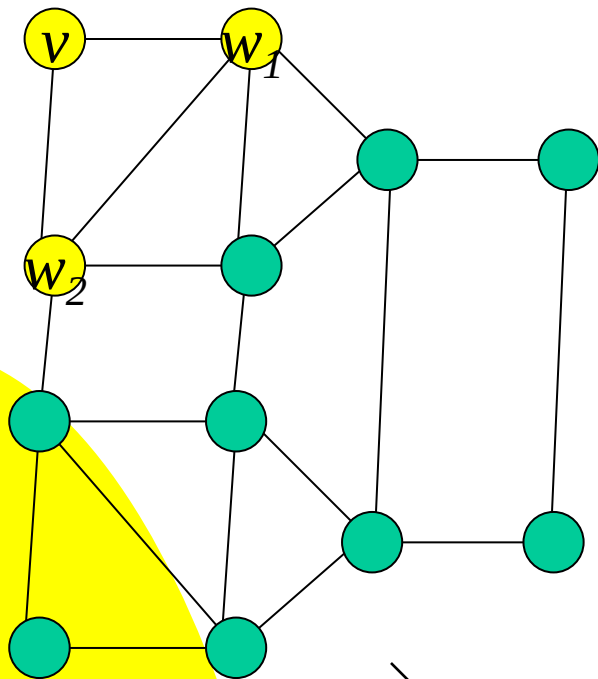
Independent Set on Planar Graphs

- NP-complete problem
- Here we argue $O(6^k n)$ algorithm
 - Faster is possible: $2^{O(\sqrt{k})} \text{poly}(n)$

Branching

- Each planar graph has a vertex of degree at most 5
- Take vertex v of minimum degree, say with neighbors $w_1, \dots, w_r$, r at most 5
- A maximum size independent set contains v or one of its neighbors
 - Selecting a vertex is equivalent to removing it and its neighbors and decreasing k by one
- Create at most 6 subproblems, one for each $x \in \{v, w_1, \dots, w_r\}$. In each, we set $k = k - 1$, and remove x and its neighbors





Analysis

- Follows from argument on previous slide
- Running time: $O(6^k n)$
- Easier algorithm
 - If $k \leq n/4$ output YES
 - Otherwise use the $O(2^n n) \leq O(2^{4k} n) = O(16^k n)$ time brute-force algorithm

Closest string

Given: k strings $s_1, \dots, s_k$ each of length L , integer d

Parameter: d

Question: is there a string s of length L with Hamming distance at most d to each of $s_1, \dots, s_k$

- Application in molecular biology
- Here: FPT algorithm
- (Gramm and Niedermeier, 2002)



Subproblems

- Subproblems have form
 - Candidate string s
 - Additional value r
 - We look for a solution to original problem, with additional condition:
 - Hamming distance at most r to s
- Start with $s = s_1$ and $r = d$ (= original problem)



Branching step

- Choose an s_j with Hamming distance $> d$ to s
 - If none exists, s is a solution
- If Hamming distance of s_j to $s > d+r$: answer *NO*
- For all positions i where s_j differs from s
 - Solve subproblem with
 - s changed at position i to value $s_j(i)$
 - $r = r - 1$
- Note: we find a solution if and only if one of these subproblems has a solution



Example

- Strings 01113, 02223, 01221, $d=2$ give as first instance
 - (01113,2)
- This branches to:
 - (02113,1)
 - (01213,1)
 - (01123,1)

Example

- Input: 01113, 02223, 01221, $d=2$
- Instance (02113,1)
 - The Hamming distance from 02113 to 01221 is 4 and $4 > 2+1 = d+r$ so the algorithm outputs NO without branching.

Example

- Input: 01113, 02223, 01221, $d=2$
- (01**2**13,1) branches to
 - (0**2**213,0) ☹️
 - (01**1**13,0) ☹️ (speedup step: skip this as it undoes an earlier change)
 - (012**2**3,0) 😊
 - (0121**1**,0) ☹️
- Etc.

Time analysis

- Recursion depth d
- At each level, we branch at most at $d + r \leq 2d$ positions
- So, number of recursive steps at most $(2d)^{d+1}$
- Each step can be done in polynomial time: $O(kdL)$
- Total time is $O((2d)^{d+1} \cdot kdL)$
- Speed up possible by more clever branching and by kernelisation



Technique

- Try to find a branching rule that
 - Decreases the parameter
 - Splits in a bounded number of subcases
 - YES, if and only if YES in at least one subcase



KERNELISATION



Universiteit Utrecht

Kernelisation

- Preprocessing rules reduce starting instance to one of size $f(k)$
 - Should work in polynomial time
- Then use any algorithm to solve problem on kernel
- Time will be $p(n) + g(f(k))$



Kernelisation

- Helps to analyze preprocessing
- Much recent research



Formal definition of kernelisation

- Let P be a parameterized problem. (Each input of the form (I,k) .)
A *reduction to a problem kernel* is an algorithm A , that transforms inputs of P to inputs of P , such that
 - $(I,k) \in P$ if and only if $A(I,k) \in P$ for all (I,k)
 - If $A(I,k) = (I',k')$, then $k' \leq f(k)$, and $|I'| \leq g(k)$ for some functions f, g
 - A uses time polynomial in $|I|$ and k



Kernels and FPT

- **Theorem.** Consider a parameterized problem. Then the problem belongs to FPT if and only if it has a kernel
 - $\Leftarrow$ Build kernel and solve the problem on kernel
 - $\Rightarrow$ Suppose we have an $f(k)n^c$ algorithm. Run the algorithm for n^{c+1} steps. If it did not yet solve the problem, return the input as kernel: it has size at most $f(k)$. If it solved the problem, then return small YES / NO instance

First kernel: Convex string recoloring

- Application from molecular biology; NP-complete
Convex String Recoloring
 - **Given:** string s in Σ^* , integer k
 - **Parameter:** k
 - **Question:** can we change at most k characters in the string s , such that s becomes *convex*, i.e., for each symbol, the positions with that symbol are consecutive.
- Example of convex string: aaacccbxxxxfff
- Example of string that is not convex: abba
- Instead of symbols, we talk about *colors*



Kernel for convex string recoloring

- **Theorem:** Convex string recoloring has a kernel with $O(k^2)$ characters.



Notions

- Notion: good and bad colors
- A color is *good*, if it is consecutive in s , otherwise it is *bad*
- abba: a is bad and b is good
- Notion of *block*: consecutive occurrences of the same color, ie. aabbacc has four blocks
- Convex: each color has one block



Construction of kernel

- Apply three reduction rules
 - Rule 1: limit #blocks of bad colors
 - Rule 2: limit #different good colors
 - Rule 3: limit #characters in s per block
- Count



Rule 1

- If there are more than $4k$ blocks of bad colors, say NO
 - Formally, transform to trivial NO-instance, e.g. (aba, 0)
 - Why correct?
 - Each change can decrease the number of blocks of bad colors by at most 4.
 - Example: ab**a**b -> abbb



Rule 2

- If we have two consecutive blocks of good colors, then change the color of the second block to that of the first
- E.g: abbbbcca -> abbbbbba
- Why correct?
 - We will only recolor the c's to connect bad colors, and this still comes at the same cost.



Example

- aaaaa**b**ccaaaaa, 4
is transformed to:
- aaaaa**bbb**aaaaa, 4

Rule 3

- If a block has more than $k+1$ characters, delete all but $k+1$ of the block
- Correctness: a block of such a size can never be changed
- **aaaaa**bbbbbaa, 2 goes to **aaa**bbbbaa, 2



Counting

- After the rules have been applied, we have at most:
 - $4k$ blocks of bad colors
 - $4k+1$ blocks of good colors: at most one between each pair of bad colors, one in front and one in the end
 - Each block has size at most $k+1$
- String has size at most $(8k+1)(k+1)$
- This can be improved by better analysis, more rules, ...



Vertex cover: observations that help for kernelisation

- Isolated vertices can be ignored
- If v has degree at least $k+1$, v belongs to each vertex cover in G of size at most k .
 - If v is not in the vertex cover, then all its neighbors are in the vertex cover.
- If all vertices have degree at most k , and $m > k^2$, there is no vertex cover of size k .



Kernelisation for Vertex Cover

$H = G; (S = \emptyset;)$

While there is a vertex v in H of degree at least $k+1$
do

Remove v and its incident edges from H

$k = k - 1; (S = S + v ;)$

If $k < 0$ then return **false**

If H has at least k^2+1 edges, then return **false**

Remove vertices of degree 0

Time

- Kernelisation step can be done in $O(n+m)$ time
- After kernelisation, we must solve the problem on a graph with at most k^2 edges, e.g., with branching this gives:
 - $O(n + m + 2^k k^2)$ time
 - $O(kn + 2^k k^2)$ time can be obtained by noting that there is no solution when $m > kn$.



Better kernel for vertex cover

- Nemhauser-Trotter: kernel of at most $2k$ vertices
- Make ILP formulation of Vertex Cover
- Solve relaxation
- All vertices v with $x_v > \frac{1}{2}$: put v in set
- All vertices v with $x_v < \frac{1}{2}$: v is not in the set
- Remove all vertices except those with value $\frac{1}{2}$, and decrease k accordingly
- Gives kernel with at most $2k$ vertices, but why is it correct?



Nemhauser Trotter proof plan

1. Write down the ILP for Vertex Cover
2. Write down the LP relaxation, let x be an optimal solution
 - Let $X_{<\frac{1}{2}} = \{v \in V : x_v < \frac{1}{2}\}$, $X_{>\frac{1}{2}} = \{v \in V : x_v > \frac{1}{2}\}$
3. Given any minimum vertex cover S , show that $S^* := (S \cup X_{>\frac{1}{2}}) \setminus X_{<\frac{1}{2}}$ also is an optimal VC.

Correctness of kernel follows from this observation



1. ILP

$$\min \sum_{v \in V} x_v$$

$$\forall \{v, w\} \in E : x_v + x_w \geq 1$$

$$x_v \in \{0,1\}$$

2. Relaxation

$$\min \sum_{v \in V} x_v$$

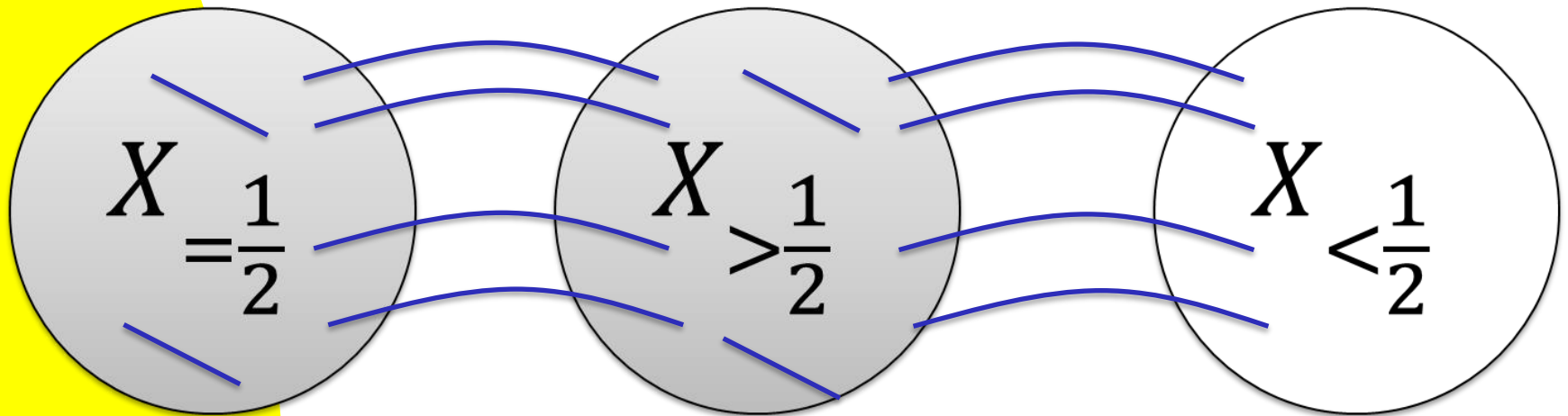
$$\forall \{v, w\} \in E : x_v + x_w \geq 1$$

$$x_v \geq 0$$

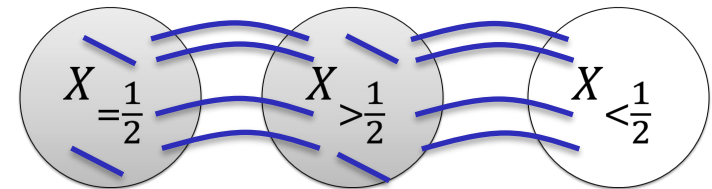
3. Observation

- If S is min. vc, so is $S^* := (S \cup X_{>\frac{1}{2}}) \setminus X_{<\frac{1}{2}}$.

$$\forall \{v, w\} \in E : x_v + x_w \geq 1$$



3. Observation



- $S^* := (S \cup X_{>\frac{1}{2}}) \setminus X_{<\frac{1}{2}}$
- Need: $|S^*| = |S| + |X_{>\frac{1}{2}} \setminus S| - |X_{<\frac{1}{2}} \cap S| \leq |S|$
- Suppose not, so $|X_{>\frac{1}{2}} \setminus S| > |X_{<\frac{1}{2}} \cap S|$
 - x is not optimal solution to relaxation:
 - Let $\varepsilon := \min_{v \in X_{>\frac{1}{2}}} x_v - 1/2$
 - Add ε to x_v if $v \in X_{<\frac{1}{2}} \cap S$; subtract ε if $v \in X_{>\frac{1}{2}} \setminus S$

$2k$ kernel for Vertex Cover

- Solve the relaxation (polynomial time with the ellipsoid method, practical with Simplex)
- If the relaxation has optimum more than k , then say no
- Otherwise, get rid of the 0's and 1's, decrease k accordingly
- At most $2k$ vertices have weight $\frac{1}{2}$ in the relaxation
- So, kernel has $2k$ vertices.
- It can (and will often) have a quadratic number of edges



Reminder: Cluster editing

- **Instance:** undirected graph $G=(V,E)$, integer k
- **Parameter:** k
- **Question:** can we make at most k modifications to G , such that each connected component is a clique, where each modification is an addition of an edge or the deletion of an edge?



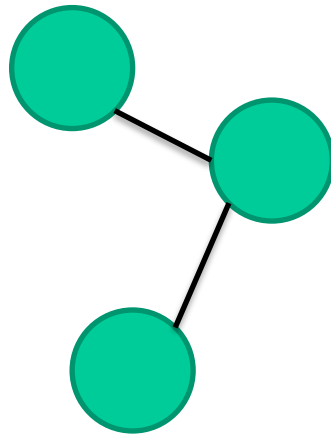
Kernelisation for cluster editing

- Quadratic kernel
- General form:
- Repeat rules, until no rule is possible
 - Rules can do some necessary modification and decrease k by one
 - Rules can remove some part of the graph
 - Rules can output YES or NO



Recall observation

- G is a cluster graph if there is no path with 3 vertices (P_3) as induced subgraph



Rule 1

- Remove a connected component that is a clique
- Safe (trivial)

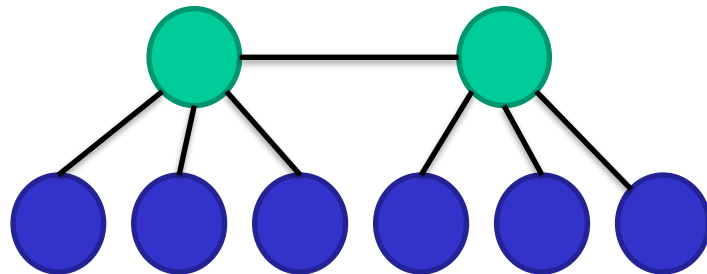
Rule 2

- If G is not a cluster graph, and $k \leq 0$, then output NO
- Trivially correct

Rule 3

- Suppose $\{v, w\}$ is an edge, and there are at least $k+1$ vertices that are adjacent to v but not to w or to w but not to v .
- Remove the edge $\{v, w\}$ and decrease k by 1

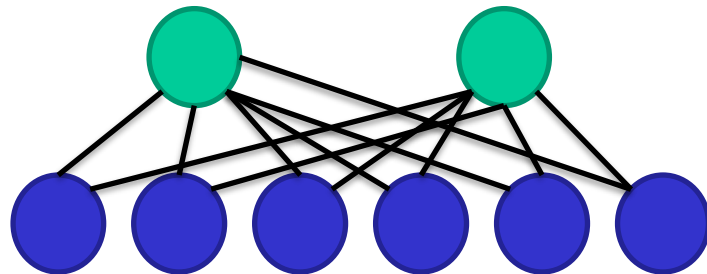
Safe: why?



Rule 4

- Suppose there is no edge from v to w .
Suppose there are at least $k+1$ vertices that are adjacent to v and to w .
- Add the edge $\{v, w\}$ and decrease k by 1.

Safe: why?



Reduction procedure

- While possible, apply rules 1 – 4
- Claim: if there is a solution, then after the reduction procedure we have at most $3k^2$ vertices
- Rule 5: If rules 1 – 4 do not apply and $|V| > 3k^2$ then output NO

Proof of claim

1. If no connected component is a clique, then each vertex is in at least one P_3
 - Consider v . If not in P_3 then all neighbors of neighbors of v are neighbors of v , so connected component is $N[v]$
2. Suppose we have a solution of k changes.
3. Each change deals with at most k P_3 's
 1. An edge removal deals with at most k P_3 's since rule 3 applies otherwise
 2. An edge addition deals with at most k P_3 's since rule 4 applies otherwise
4. Hence there are at most k^2 P_3 's and $3k^2$ vertices



Result and remark

- We have a kernel with quadratic number of vertices
- Kernels with linear number of vertices exist



Concluding remarks

- Similar techniques work (usually much more complicated) for many other problems
- Note similarities and differences with exponential time algorithms

